



Tecnológico de Monterrey

Reto Selección, configuración y entrenamiento del modelo

Luisa Merlo García | A01067715

Edgar Osvaldo Navarro | A01644488

Ariana Guadalupe Rosales Villalobos | A01644773

Inteligencia Artificial Avanzada para la Ciencia de Datos (601)

Profesor Gildardo Sánchez Ante

Introducción

El dataset de REHAB recolecta datos sobre pacientes sobrevivientes de un Accidente Cerebrovascular. Consiste en 32 archivos los cuales representan los datos recolectados por 2 distintos conjuntos de sensores al momento de realizar 16 movimientos de rehabilitación. Estos datos son mayormente series temporales que demuestran el estado de distintos canales de los sensores al momento de realizar el movimiento. Cada movimiento cuenta con un total de 11 canales y estos recolectaron 880 puntos a una frecuencia de 50 hz correspondiendo a 17.6 segundos en caso de que el movimiento se hubiera terminado antes de tiempo se llenaron los puntos faltantes con 0. A partir de este dataset se busca predecir qué tipo de movimiento se está haciendo referencia utilizando estas series de tiempo. A continuación se presenta el proceso realizado para tratar los datos partiendo desde los datos raw y la implementación del algoritmo del centroide más cercano utilizando ventanas temporales para reemplazar las series de tiempo.

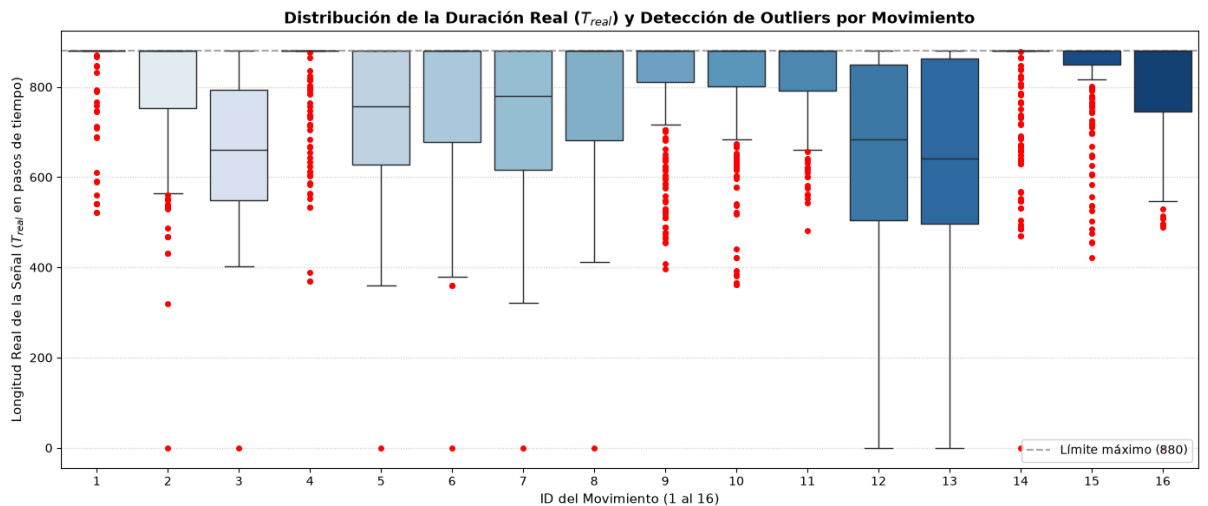
Tratamiento de datos

Búsqueda y reemplazo de duplicados, datos nulos y muestras corruptas

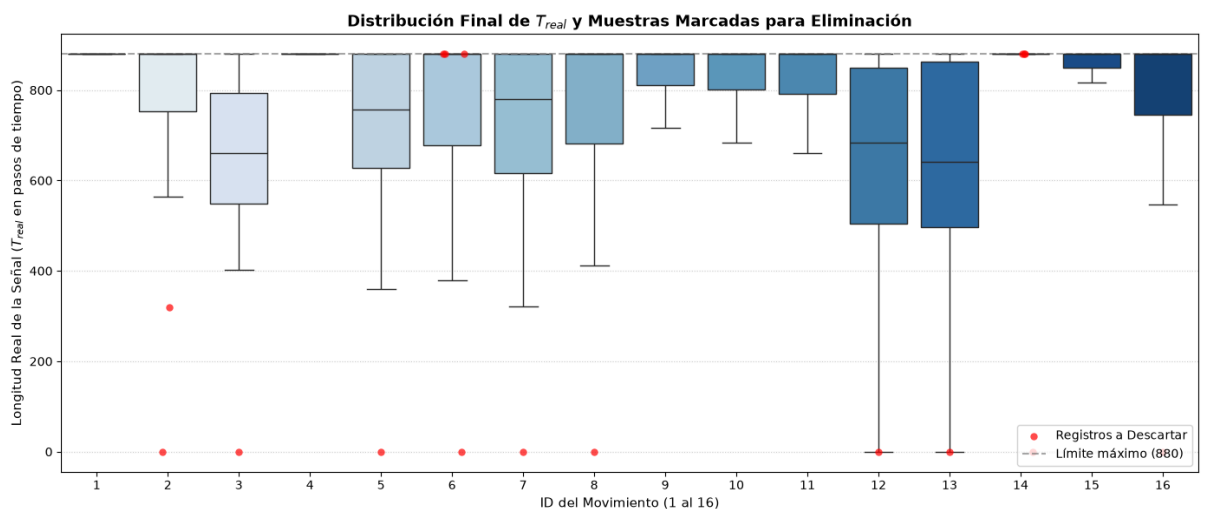
Lo primero que se realizó fue probar que los tamaños de las muestras a utilizar tuvieran las mismas dimensiones y que se tuvieran el mismo tamaño de muestra para los

distintos sensores. Afortunadamente se tenían las mismas cantidades de muestras y no se requirió realizar un tratamiento para esta parte. Después se revisaron todos los datasets de los archivos en búsqueda de nulls o infinitos. Esto nos dio como resultado que no existe ningún dato nulo o infinito. Utilizando numpy se buscó en cada archivo el número de registros duplicados para los cuales primero se concatenaron los datos de ambos grupos de sensores y después se realizó la búsqueda teniendo un total de 716 registros duplicados entre todos los movimientos representando el 7.76% de los registros guardados. En este caso al estos ser registros duplicados se decidió eliminarlos del dataframe.

Pero también es necesario checar la proporción de ceros dentro de nuestros archivos al haber sido utilizados para agregar un padding y existe la probabilidad de encontrar canales muertos con ellos. Para esto se contaron el número de ceros por canal y se contaron un total de 6153769 representando al 89.65% de los datos registrados en las series de tiempo. En este caso el número es tan grande debido a que pudieron haber registros muertos o con algún problema en algún sensor, y también por el padding agregado para hacerlos todos series de tiempo de 880 puntos. Para arreglar este problema se revisaron las varianzas de todos los registros y se encontraron 16 registros que no tenían casi ninguna varianza lo cual fue clasificado como un registro con canal muerto y posteriormente eliminado. A su vez se graficaron las longitudes reales de los movimientos en un boxplot para revisar si había datos fuera de lo común que pudiera afectar de gran manera al modelo.



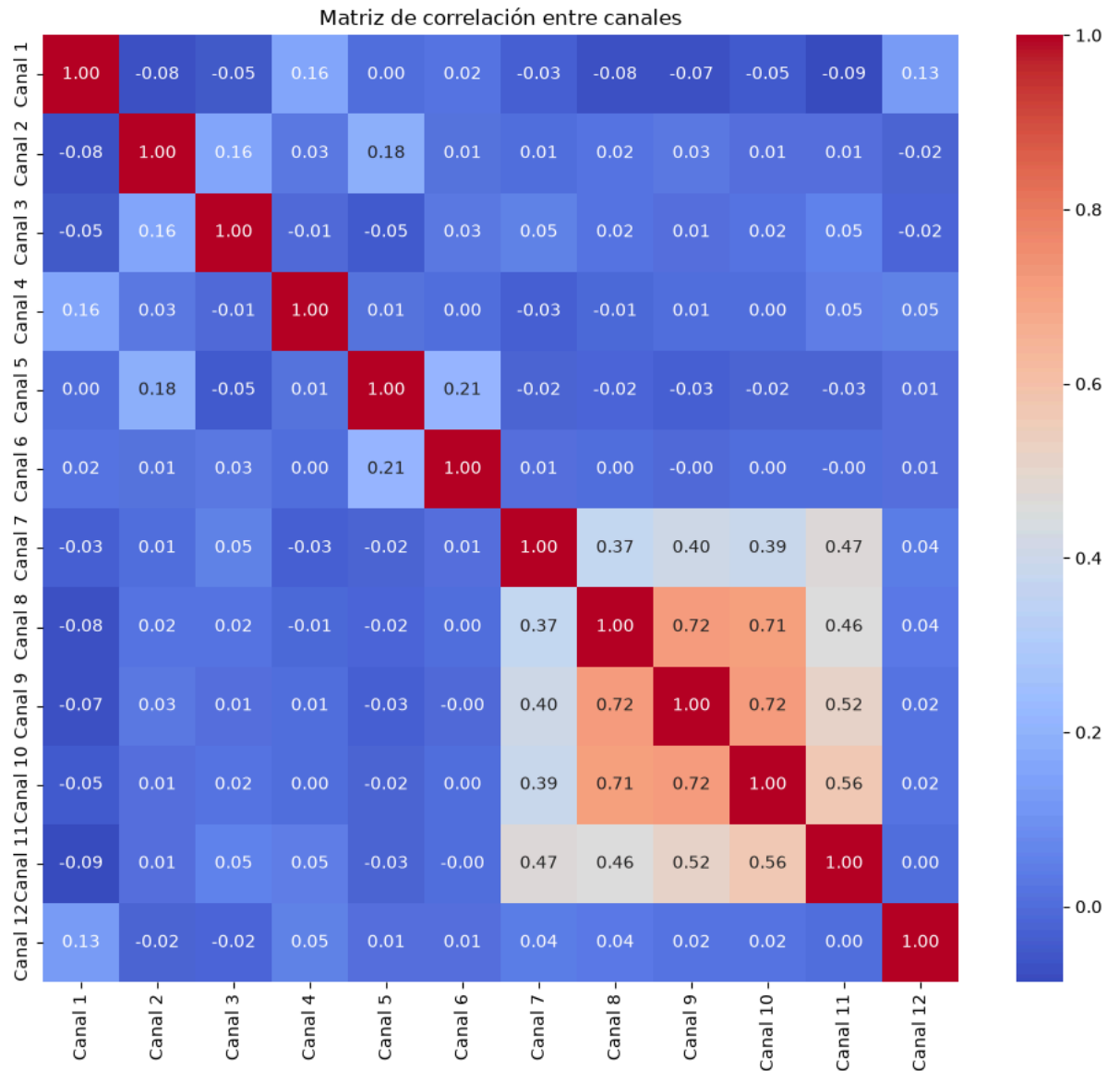
A Través de esta gráfica se observó, que existían varios movimientos con muchos valores atípicos utilizando un IQR normal y en este caso eliminarlos podría significar un gran desbalance en nuestro dataset. Por lo tanto, se decidió utilizar un IQR modificado para detectar únicamente casos muy extremos en los cuales están por debajo del 40% de la mediana. En este caso se detectó únicamente un valor el cual cumplía con esta condición, después se graficaron estos puntos en un boxplot junto con los registros a eliminar por canal muerto.



Transformación de datos

Para empezar la transformación, lo primero que se realizó fue checar el tamaño de la muestra en los distintos movimientos. Afortunadamente se pudo observar que las variaciones no fueron muy altas siendo el movimiento con más muestras el movimiento 8 con 383 y el de menos muestras el movimiento 3 con 171. Tras realizar esto se decidió unir todos los sensores en un solo data frame y separar sus etiquetas obteniendo un dataset de (3883, 880, 12) para los datos y otro de (3883) con las etiquetas.

Después se revisaron los estadísticos básicos considerando el padding de ceros. Para considerarlo tan solo se tomaron en cuenta los valores de la muestra hasta que se detecta que tienen ese padding considerando únicamente la porción del movimiento captado. Viendo las estadísticas se decidió normalizar los datos para poder tener una varianza de 1 y poder entrenar el modelo de mejor manera. También se analizó la correlación entre las variables obteniendo el siguiente mapa de calor:



A través de este mapa de calor se puede observar que los canales del segundo grupo de sensores estaba muy correlacionado lo cual puede significar que estos canales tienen medidas muy similares. Por esta razón se decidió unir los canales 8, 9 y 10 en uno solo canal promediado obteniendo un dataset de (3883, 880, 10).

Modelos de clasificación

Identificación

Dado que el reto consiste en predecir a cuál de los 16 movimientos de rehabilitación corresponde un registro, se identificó que el problema requiere un modelo de clasificación multiclase supervisado. Los datos de entrada son series de tiempo multivariadas capturadas por dos conjuntos de sensores, por lo que antes de entrenar cualquier modelo fue necesario transformarlas en vectores de características de tamaño fijo, ya que la mayoría de los algoritmos de clasificación clásicos no aceptan secuencias de longitud variable como entrada directa. Por esta razón se descartaron modelos que requieren datos puramente secuenciales (como RNN o LSTM) y en su lugar se investigaron y compararon distintos algoritmos de clasificación tabular compatibles con vectores numéricos de longitud fija: Regresión Logística, K-Vecinos más Cercanos (KNN), Máquinas de Vectores de Soporte (SVM con kernel RBF), *Random Forest*, *Extra Trees* y *HistGradientBoosting*. Estos modelos fueron seleccionados porque son ampliamente utilizados para clasificación multiclase con datos tabulares, cubren un espectro amplio de complejidad (desde modelos lineales hasta ensambles no lineales) y permiten comparar el efecto de la complejidad del modelo sobre el desempeño en un dataset de tamaño moderado (3,883 registros, aproximadamente 240 por clase).

Configuración y entrenamiento

Para preparar los datos de entrada de los modelos, la línea de tiempo de 880 pasos de cada canal se dividió en 11 ventanas de tamaño dinámico según la duración real del movimiento (sin contar el padding de ceros). De cada ventana se extrajeron 5 estadísticos por canal (media, desviación estándar, mínimo, máximo y RMS), lo que convirtió cada registro

en un vector de 550 características ($10 \text{ canales} \times 11 \text{ ventanas} \times 5 \text{ estadísticos}$). El conjunto de datos fue dividido en tres subconjuntos, con: 60% para entrenamiento, 20% para validación y 20% para prueba. El conjunto de entrenamiento se utilizó para ajustar los parámetros internos del modelo, mientras que el conjunto de validación se reservó para la selección de hiper-parámetros. Mientras que el conjunto de pruebas se mantuvo separado y se utilizó únicamente para la evaluación final.

Para realizar el ajuste de hiper parámetros se utilizó *GridSearchCV* utilizando *GridSearchCV* con la función de Sklearn *PredefinedSplit*, lo que permitió separar explícitamente que observaciones correspondían al conjunto de entrenamiento, cuáles de validación y de prueba. Para cada combinación de hiperparametros, el modelo se entrenó con los datos de entrenamiento y se evaluó sobre validación utilizando F1 Macro como métrica de selección al mismo tiempo que se utilizó balanced Accuracy para validar el desempeño entre clases, debido al desbalance de muestras entre las 16 clases. Una vez que se seleccionaba la mejor configuración, se reentrenaba el modelo con entrenamiento y validación. Una vez sacadas las métricas podíamos determinar el rendimiento de cada modelo antes y después del uso de los datos del conjunto de prueba.

Resultados y conclusiones

De los modelos entrenados, los ensambles basados en árboles obtuvieron el mejor desempeño: Extra Trees logró el F1 macro más alto en prueba (0.9567) con una exactitud de 96.14%, seguido muy de cerca por *HistGradientBoosting* (F1 macro 0.9550, exactitud 95.75%) y Random Forest (F1 macro 0.9495, exactitud 95.50%). SVM RBF y KNN quedaron en un nivel intermedio (F1 macro de 0.9289 y 0.9285, respectivamente), mientras

que la Regresión Logística obtuvo el desempeño más bajo entre los modelos optimizados con *GridSearchCV* (F1 macro 0.8655, exactitud 87.39%).

Se seleccionó Extra Trees como modelo final debido a que combina el mejor desempeño en validación cruzada (F1 macro de validacion 0.9590) con el mejor desempeño en el conjunto de prueba, mostrando además un comportamiento consistente entre ambas métricas, sin señales de sobreajuste. Este resultado tiene sentido dado que los ensambles de árboles no requieren que las 550 características extraídas sean linealmente separables ni sigan una distribución particular, y son capaces de capturar relaciones no lineales entre los estadísticos de distintos canales y ventanas temporales. Se evaluaron 24 combinaciones de hiperparametros, correspondientes a diferentes valores de números de árboles, profundidad máxima, número mínimo de pruebas para realizar una división y número de características consideradas en la división. La mejor configuración obtenida en este caso corresponde a 400 árboles, sin máximo de profundidad, un mínimo de dos muestras para permitir una división de raíz cuadrada como número de características candidatas a división.

Estos modelos son menos sensibles a la redundancia entre canales que se había detectado previamente en la matriz de correlación (por ejemplo, entre los canales 8, 9 y 10 del segundo sensor), y su naturaleza de ensamble reduce la varianza que un solo árbol tendría con un espacio de características de 550 dimensiones.

En contraste, la Regresión Logística obtuvo el peor desempeño entre los modelos con *GridSearchCV* porque, al ser un modelo lineal, no puede capturar relaciones no lineales entre los estadísticos temporales de los distintos canales, que es precisamente donde se distinguen movimientos cinemáticamente similares. SVM y KNN mejoraron respecto a la regresión logística al poder modelar fronteras de decisión no lineales, pero su desempeño se ve limitado

por la maldición de la dimensionalidad: con 550 características, las distancias entre muestras (de las que dependen ambos algoritmos) se vuelven menos informativas, lo cual generalmente requiere más datos de entrenamiento de los que dispone el dataset (3,106 muestras) . Por estas razones se concluye que, para este problema, los modelos de ensamble de árboles particularmente Extra Trees, son la mejor opción, al lograr el mejor puntaje F1 macro en el conjunto de validación y en el conjunto de prueba, esto es razonable ya que Extra Trees se destaca por el manejo de características redundantes y modelado de relaciones no lineales en los datos.